



SAPIENZA
UNIVERSITÀ DI ROMA

Oltre il Gold Standard: analisi comparativa tra metriche tradizionali e approcci LLM-as-a-Judge per la valutazione del web parsing

Facoltà di Ingegneria dell'Informazione, Informatica e Statistica
Ingegneria Informatica e Automatica

Ludovica Fiocchetta

Matricola 2103669

Relatore

Professore Roberto Navigli

Anno Accademico 2025/2026

**Oltre il Gold Standard: analisi comparativa tra metriche tradizionali e approcci
LLM-as-a-Judge per la valutazione del web parsing**

Laurea Triennale. Sapienza Università di Roma

© 2026 Ludovica Fiocchetta. Tutti i diritti riservati

Questa tesi è stata composta con $\text{\LaTeX}$ e la classe Sapthesis.

Email dell'autore: fiocchetta.2103669@studenti.uniroma1.it

Indice

1	Introduzione	1
1.1	Contesto generale	1
1.2	Motivazione: il progetto Chat Minerva	2
1.3	Obiettivi e contributo dell'elaborato	2
1.4	Struttura della tesi	3
2	Lavori correlati	5
2.1	Tecniche di estrazione	5
2.2	Metriche tradizionali	5
2.3	Metriche semantiche	5
2.4	LLM-as-a-Judge	5
3	Sistema presentato: architettura e implementazione	7
3.1	Panoramica dell'infrastruttura	7
3.2	Progettazione del Backend	8
3.2.1	Gestione degli endpoint	9
3.2.2	Logica dei parser	11
3.3	Progettazione del Frontend	13
3.4	Gestione dei Dati e Persistenza	13

Capitolo 1

Introduzione

1.1 Contesto generale

Nell'attuale contesto informativo tecnologico, il World Wide Web rappresenta il più vasto bacino di conoscenza non strutturata mai esistito. Con lo sviluppo dei Large Language Models, l'abilità di elaborare e comprendere il linguaggio naturale è diventata una necessità, poiché l'efficacia di questi modelli è fortemente dipendente dalla qualità dei dati forniti in input. Gli LLM hanno ancora una capacità limitata di manipolare con precisione la conoscenza, infatti falliscono nell'accedere a fonti precise e tendono ad avere allucinazioni su compiti *knowledge-intensive*. Le sfide più note includono:

- Presentazione di informazioni false quando non si ha la risposta;
- Presentazione di informazioni obsolete o generiche quando l'utente si aspetta una risposta specifica e attuale;
- Creazione di una risposta da fonti non autorevoli;
- Creazione di risposte imprecise a causa della confusione terminologica, in cui diverse fonti di addestramento utilizzano la stessa terminologia per parlare di argomenti diversi.

Per questi motivi il processo di estrazione di dati puliti assume un ruolo fondamentale. Nel paradigma della *Retrieval-Augmented Generation* (RAG) [1], architettura che ottimizza le prestazioni di un modello di intelligenza artificiale collegandolo a knowledge base esterne, il *web scraping* tradizionale, basato su regole deterministiche e selettori del Document Object Model (DOM), risulta spesso fragile di fronte alla grande diversità delle strutture HTML. Questo approccio spesso lascia insieme al testo utile anche "rumore", come menu di navigazione, banner pubblicitari e riferimenti ad altre pagine, difetto che può tradursi in allucinazioni nelle risposte degli LLM. Un approccio più robusto richiede sistemi di *web parsing* semantico, che consentono di migliorare i risultati essendo in grado di scansionare grandi database e di isolare esclusivamente le informazioni rilevanti nella pagina, ignorando tutti gli elementi di disturbo.

1.2 Motivazione: il progetto Chat Minerva

Questa necessità operativa si manifesta concretamente all'interno del progetto Chat Minerva sviluppato da Sapienza Università di Roma, per il quale è indispensabile fornire al modello testi in input totalmente puliti al fine di garantire risposte attendibili. Chat Minerva è un modello linguistico avanzato che si pone l'obiettivo di fornire un agente di chat affidabile, capace di rispondere in modo accurato e contestualizzato alle molteplici domande da parte degli utenti. Si basa su un'architettura *Retrieval-Augmented Generation* (RAG), così da fare riferimento a una base di conoscenza autorevole al di fuori dei suoi dati di addestramento originali prima di generare una risposta [2], il cui flusso operativo è illustrato in Figura 1.1.

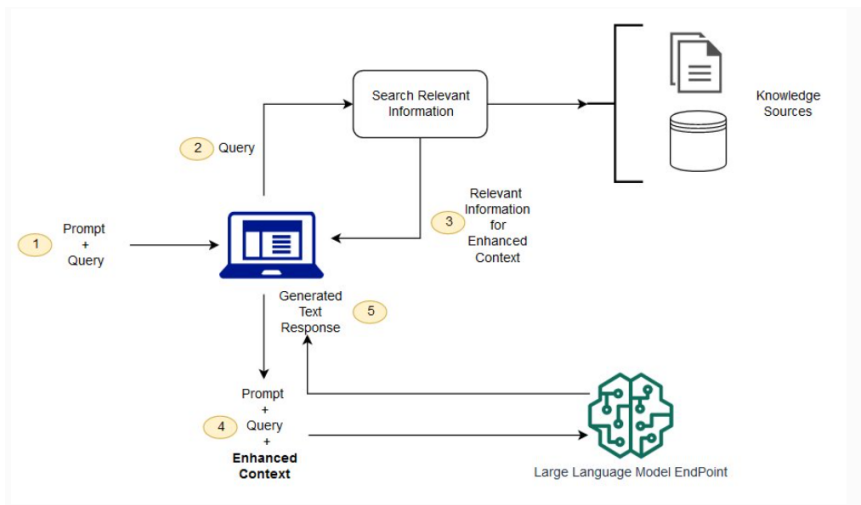


Figura 1.1. Rappresentazione di un'architettura RAG. Il framework sviluppato assicura la qualità dei dati estratti prima del loro inserimento nelle *Knowledge Sources* [2].

Come evidenziato dallo schema, quando l'utente formula una query (passaggi 1 e 2), il sistema ricerca informazioni pertinenti all'interno delle proprie fonti di conoscenza esterne per arricchire il contesto del prompt (passaggi 3 e 4). Se i parser che alimentano questi database estraggono testo sporco o frammenti di codice HTML, l'agente genererà risposte errate o allucinate (passaggio 5). Affinché il modello non generi informazioni sbagliate o non pertinenti è essenziale che i dati forniti in fase di inferenza siano estremamente puliti. Se i parser utilizzati per alimentare Minerva estraggono testo sporco o formattazioni residue, la qualità della risposta del modello degrada drasticamente. È nata quindi la necessità ingegneristica di sviluppare un'infrastruttura capace di testare, misurare e validare in modo oggettivo le performance di diversi sistemi di web parsing, garantendo che al Chatbot venga passata esclusivamente la conoscenza utile e strutturata di cui ha bisogno.

1.3 Obiettivi e contributo dell'elaborato

Il seguente elaborato ha dunque come scopo principale la descrizione di un sistema di estrazione automatizzata di testo pulito da quattro domini internet, e la succes-

siva analisi comparativa sulla valutazione della qualità dei parser. Questo studio metterà a confronto le metriche matematiche tradizionali, che basano il giudizio sulla comparazione con un *gold standard*, con l'innovativo paradigma *LLM-as-a-Judge*, esplorato anche in scenari *reference-free*. Nello specifico, il sistema si articola nei seguenti punti:

- **Sviluppo della pipeline di estrazione e valutazione:** implementazione di un'infrastruttura Python capace di estrarre testo pulito e misurare in modo oggettivo le performance dei sistemi di *web parsing*.
- **Analisi con metriche classiche e avanzate:** valutazione dei *parser* basata sul confronto dei testi con un *gold standard*, calcolando metriche rigorose quali Precision e Recall, ma anche semantiche come BERTScore e METEOR.
- **Sperimentazione del paradigma LLM-as-a-Judge:** introduzione di un approccio moderno in cui un Large Language Model agisce da giudice, esplorando l'efficacia di valutazioni *reference-free* per superare la rigidità delle metriche tradizionali.

Il contributo principale dell'elaborato risiede proprio in questo doppio livello della pipeline di valutazione. Infatti, da un lato si ha un giudizio basato su un *gold standard*, in cui il contenuto estratto viene confrontato con un testo di riferimento salvato all'interno di un database relazionale. In questa fase vengono calcolate metriche classiche basate sulla sovrapposizione insiemistica e sulle distanze operative (come la *Jaccard Similarity*), affiancate da metriche semantiche avanzate (come BERTScore e METEOR). Dall'altro lato, si esplora l'approccio *LLM-as-a-Judge*, chiamando un modello linguistico per valutare la coerenza e la pulizia del testo estratto anche in modalità *reference-free*. Questo permette di superare il limite intrinseco del *gold standard*, che richiede una costosa annotazione manuale preventiva. L'LLM agisce come giudice autonomo: attraverso tecniche di *prompt engineering* e pipeline difensive contro i colli di bottiglia computazionali, è in grado di analizzare la pulizia dell'estrazione anche su domini web sconosciuti, fornendo un punteggio quantitativo e un commento qualitativo pronto all'uso. Questo studio dimostrerà come l'integrazione di approcci semantici avanzati sia essenziale per certificare la qualità del testo estratto, garantendo l'affidabilità di sistemi complessi come Chat Minerva.

1.4 Struttura della tesi

La trattazione è organizzata come segue:

- **Capitolo 2:** analizza i lavori correlati, descrivendo l'evoluzione delle tecniche di *web parsing* e presentando il fondamento teorico delle metriche di valutazione impiegate nel progetto, dagli approcci tradizionali e semantici fino all'introduzione del paradigma *LLM-as-a-Judge*.
- **Capitolo 3:** descrive in dettaglio la progettazione architetturale del sistema, illustrando dettagliatamente tutte le scelte informatiche adottate.

- **Capitolo 4:** esplora l'implementazione degli approcci *LLM-as-a-Judge*, analizzando la costruzione dei *prompt* di valutazione e l'integrazione con i modelli in esecuzione locale.
- **Capitolo 5:** presenta l'impostazione degli esperimenti sui domini selezionati e discute criticamente i risultati ottenuti, confrontando i punteggi delle metriche deterministiche con il giudizio qualitativo dell'intelligenza artificiale.
- **Capitolo 6:** riassume i traguardi raggiunti e delinea i possibili sviluppi futuri per il potenziamento dell'ecosistema.

Capitolo 2

Lavori correlati

2.1 Tecniche di estrazione

2.2 Metriche tradizionali

2.3 Metriche semantiche

2.4 LLM-as-a-Judge

Capitolo 3

Sistema presentato: architettura e implementazione

3.1 Panoramica dell'infrastruttura

Questo capitolo descrive nel dettaglio l'architettura ingegneristica alla base del framework di valutazione. Il sistema ha come obiettivo principale l'estrazione automatizzata di testo pulito dalle pagine di quattro domini internet tramite la libreria *Crawl4AI* e la successiva valutazione dell'accuratezza dei parser, mediante il confronto tra il testo estratto dal *crawler* e il *gold standard* di riferimento creato manualmente. I domini di riferimento sono:

- `en.wikipedia.org`;
- `viaggi-usa.it`;
- `cbsnews.com`;
- `cnbc.com`.

Il progetto è strutturato seguendo un'architettura a microservizi, separando la logica di estrazione, valutazione e salvataggio dei dati (*Backend*) dall'interfaccia utente (*Frontend*). L'intero sistema è containerizzato mediante Docker, tramite le specifiche definite nel file `docker-compose.yaml` per garantire la portabilità e riproducibilità dell'ambiente di esecuzione. Docker è infatti una piattaforma *open source* per lo sviluppo, la distribuzione e l'esecuzione delle applicazioni. Consente di separare queste ultime dall'infrastruttura grazie all'utilizzo dei *container*, ambienti leggeri e isolati che racchiudono tutto il necessario per eseguire un'applicazione in modo indipendente dal sistema host [3]. Nello specifico del framework, l'orchestrazione tramite Compose ha permesso di avviare simultaneamente e far comunicare i moduli di estrazione, il database relazionale, il modello linguistico locale e l'interfaccia utente.

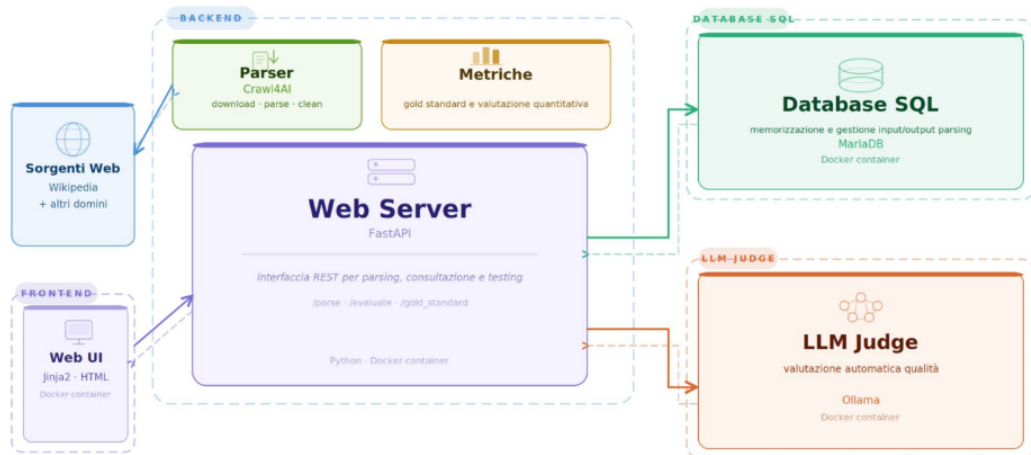


Figura 3.1. Architettura a microservizi del progetto. Lo schema illustra la comunicazione tra le varie componenti del sistema.

3.2 Progettazione del Backend

Il *backend*, esposto sulla porta 8003, è stato sviluppato seguendo un'architettura modulare in Python e si articola nei seguenti macro-componenti fondamentali:

- **Validazione dati e server (`server.py`):** questo modulo espone i servizi verso il *frontend* sfruttando il framework FastAPI. Implementa l'interfaccia REST API e utilizza modelli Pydantic per una validazione rigorosa dei dati in ingresso e in uscita, assicurando che ogni endpoint (`/parse`, `/domains`, `/gold_standard`, ecc.) rispetti le specifiche richieste. La logica di *routing* smista dinamicamente le richieste ai diversi *parser* in base al dominio dell'URL fornito.
- **Motore di estrazione:** ognuno dei quattro domini è gestito da un modulo dedicato che implementa una *pipeline* di *parsing* a tre stadi basata sulla libreria Crawl4AI:
 1. *Configurazione e acquisizione:* viene utilizzato `BrowserConfig` in modalità *headless* e `CrawlerRunConfig` con `CacheMode.BYPASS` per garantire l'acquisizione in tempo reale del contenuto. In alternativa, fornendo in input una stringa HTML, permette l'estrazione statica diretta senza interrogazione web.
 2. *DOM cleaning tramite JavaScript:* prima della conversione in Markdown, viene iniettato uno script JS mirato per manipolare il DOM e rimuovere selettivamente elementi di disturbo come infobox, sidebar, footer e banner.
 3. *Post-processing regex:* il testo convertito in Markdown subisce un ulteriore filtraggio tramite espressioni regolari per eliminare residui di tag, metadati, riferimenti bibliografici e spazi bianchi ridondanti, restituendo un testo pulito pronto per l'analisi.

- **Calcolo delle metriche (*accuracy.py*):** rappresenta il motore di valutazione dell'intero sistema. È stato progettato come un servizio indipendente che riceve stringhe di testo (estratto e di riferimento) e calcola il grado di sovrapposizione tra i set di token, operando una normalizzazione preventiva (*lowercase* e rimozione della punteggiatura) per garantire l'oggettività del confronto.

Completano l'architettura *backend* gli *script* ausiliari `init_db.py` per l'inizializzazione della base di dati, `remove_markdown.py` per la rimozione finale del Markdown e `clean_html.py` per la pulizia dell'html grezzo necessaria all'*LLM-as-a-Judge* in configurazione *Reference-Free*.

3.2.1 Gestione degli endpoint

Il cuore comunicativo del *backend* è gestito da FastAPI, un framework *web* moderno e ad alte prestazioni per la creazione di API in Python, basato sulla tipizzazione standard *type hints*. I principali punti di forza di questo strumento risiedono nella sua robustezza, nella facilità di sviluppo e nella capacità di generare automaticamente una documentazione interattiva [4]. Il framework espone i servizi attraverso un'architettura RESTful e garantisce la validazione di ogni richiesta e risposta tramite i modelli *Pydantic*. Questi ultimi prevengono la creazione di dati malformati imponendo formati specifici (come `ParseRequest` o `EvaluateRequest`) e assicurano la generazione di JSON per una facile integrazione con gli altri componenti del sistema [5].

Le rotte dell'API possono essere raggruppate in tre macro-categorie:

- **Servizi di estrazione (*/parse*):** l'endpoint principale accetta richieste POST contenenti l'URL da pulire. Il dominio viene analizzato e instradato dinamicamente al *parser* di riferimento per l'elaborazione. Una variabile booleana consente di verificare preliminarmente la presenza dell'HTML nel database locale: se `local=True` il *parsing* avviene off-line, riducendo drasticamente i tempi di latenza e le richieste di rete, altrimenti viene scaricato l'HTML originale.
- **Servizi di valutazione (*/evaluate* e */evaluate_judge*):** questi endpoint costituiscono il motore di analisi quantitativa. */evaluate* esegue la pulizia del Markdown (`remove_markdown`) e restituisce le metriche deterministiche e semantiche calcolate dal modulo `accuracy.py`. Le rotte */evaluate_judge* e */evaluate_judge_no_gs* instaurano invece una comunicazione asincrona, tramite la libreria `httpx` [6], con il container di Ollama, interrogando l'LLM con i *prompt* opportunamente ingegnerizzati e imponendo al modello di restituire un output rigorosamente in formato JSON, contenente il voto numerico e il *feedback* qualitativo.
- **Gestione dello stato e statistiche (*/db_stats*, */full_gs_eval*):** un set di endpoint dedicati al monitoraggio e all'aggregazione dei dati. Permettono al *frontend* di calcolare le medie delle metriche per ogni dominio, interrogare e salvare i dati nel database e gestire le operazioni CRUD (Create, Read, Update, Delete) sulle risorse web memorizzate e sui *gold standard*. Completa il

modulo la rotta `/status` per il controllo dell'attività del *backend*, dell'integrità del *database* e della raggiungibilità del modello linguistico.

Di seguito è riportato un frammento di codice che mostra i modelli di validazione, inclusa la struttura annidata della risposta e l'endpoint dedicato al calcolo delle metriche di valutazione:

```

1  [language=Python, caption={Definizione dei modelli Pydantic e dell'
    endpoint per la valutazione quantitativa.}, label={lst:
    evaluate_endpoint}]
2  # Modello per le metriche standard
3  class TokenLevelEval(BaseModel):
4      precision: float
5      recall: float
6      f1: float
7  # Modello per le altre metriche
8  class XEval(BaseModel):
9      jaccard_similarity: float
10     bigram_overlap: float
11     cosine_similarity: float
12     meteor: float
13     bert_score: float
14
15 # Modelli di Input/Output per l'endpoint /evaluate
16 class EvaluateRequest(BaseModel):
17     parsed_text: str
18     gold_text: str
19
20 class EvaluateResponse(BaseModel):
21     token_level_eval: TokenLevelEval
22     x_eval: XEval
23
24 # Endpoint per il calcolo delle metriche
25 @app.post("/evaluate", response_model=EvaluateResponse)
26 async def evaluate_text(data: EvaluateRequest):
27
28     # Rimozione del Markdown prima della valutazione
29     normalized_parsed = remove_markdown(data.parsed_text)
30     normalized_gold = remove_markdown(data.gold_text)
31
32     # Chiamata al motore di calcolo indipendente
33     ris = calculate_metrics(normalized_parsed, normalized_gold)
34
35     return {
36         "token_level_eval": TokenLevelEval(
37             precision=ris["precision"],
38             recall=ris["recall"],
39             f1=ris["f1"]
40         ),
41         "x_eval": XEval(
42             jaccard_similarity=ris["jaccard_similarity"],
43             bigram_overlap=ris["bigram_overlap"],
44             cosine_similarity=ris["cosine_similarity"],
45             meteor=ris["meteor"],
46             bert_score=ris["bert_score"]
47         )
48     }

```

3.2.2 Logica dei parser

Come anticipato nella panoramica, la fase di acquisizione e pulizia dei dati è affidata a script specializzati per singolo dominio, implementati sfruttando la libreria asincrona *Crawl4AI*. Sebbene i selettori CSS e le espressioni regolari varino a seconda della struttura sorgente, l'impianto logico della *pipeline* rimane invariato.

L'estrazione si articola in tre fasi sequenziali:

1. **Configurazione e acquisizione:** impostazione del *browser headless* e del generatore Markdown, con regole restrittive per ignorare link e immagini al fine di ridurre il rumore.
2. **Manipolazione del DOM (Pre-processing):** iniezione di codice JavaScript eseguito direttamente nel *browser* prima dell'estrazione. Questo passaggio è cruciale per rimuovere alla radice elementi strutturali non informativi (come *sidebar*, *footer* e banner pubblicitari) prima che vengano convertiti in testo.
3. **Filtraggio tramite regex (post-processing):** raffinamento finale sul testo Markdown generato, per eliminare scorie residue, tag di formattazione o metadati non catturati dai selettori CSS.

A titolo esemplificativo, si riporta nel Listato ?? l'implementazione del *parser* dedicato al dominio di Wikipedia. Il codice evidenzia la logica di rimozione degli *infobox* tramite JavaScript e la successiva pulizia dei tipici tag redazionali (es. `[edit]`, `[citation needed]`) tramite espressioni regolari.

```

1 [language=Python, caption={Implementazione del \textit{parser}
   asincrono per il dominio Wikipedia.}, label={lst:parser_wiki}]
2 from crawl4ai import AsyncWebCrawler, BrowserConfig, CrawlerRunConfig
   , CacheMode, DefaultMarkdownGenerator
3 import re
4
5 async def parser_wiki(url: str, html_text: str) -> dict:
6
7     # Configurazione del browser
8     browser_config = BrowserConfig(headless=True)
9
10    # Configurazione del Markdown Generator
11    md_generator = DefaultMarkdownGenerator(
12        options={
13            "ignore_links": True,
14            "escape_html": False,
15            "ignore_images": True}
16    )
17
18    # Iniezione JavaScript per la pulizia del DOM
19    remove_infobox_js = """
20        const infoboxes = document.querySelectorAll('.infobox, .
21        sidebar,
22        .vertical-navbox, .navbox, .portal, .toc, .metadata, .ambox,
23        .hatnote, .shortdescription, .figcaption, .thumbcaption');
24        infoboxes.forEach(box => box.remove());
25        const figures = document.querySelectorAll('figure');
26        figures.forEach(f => f.remove());

```

```

26     """
27
28     css_list = ["h1#firstHeading", "#mw-content-text"]
29
30     # Configurazione del Crawler
31     crawler_config = CrawlerRunConfig(
32         cache_mode=CacheMode.BYPASS,
33         target_elements=css_list,
34         markdown_generator=md_generator,
35         js_code=remove_infobox_js
36     )
37
38     # Esecuzione del crawler (da web o da HTML locale)
39     async with AsyncWebCrawler(config=browser_config) as crawler:
40         if html_text == '':
41             result = await crawler.arun(url=url, config=
42 crawler_config)
43         else:
44             result = await crawler.arun(url=f"raw:{html_text}",
45 config=crawler_config)
46
47     # Pulizia del testo
48     clean_text = re.sub(r"[[a-z]]\\[[d+\\]\\[edit\\]\\[show\\]\\[
49 update\\]
50 /\\[s*\\]", "", result.markdown)
51     clean_text = re.sub(r'(\s*/_)(.*?)\1', r'\2', clean_text)
52     clean_text = re.sub(r"[[citation needed]\\[clarification needed
53 \\]
54 /\\[supporting\\]\\[[A-Z]+\\]", "", clean_text)
55     clean_text = re.sub(r"^[^/:s/]+\\n?", "", clean_text,
56 flags=re.MULTILINE)
57     clean_text = re.sub(r'^.*Main article[s]?:\s*.*$\n?', '',
58 clean_text, flags=re.MULTILINE | re.IGNORECASE)
59     clean_text = re.sub(r"\s*\\[s*", " ", clean_text)
60     clean_text = re.sub(r"\s*/\s*", " ", clean_text)
61     text = re.split(r"##\s*(?:See also/References/Notes/Further
62 reading
63 /External links)", clean_text, flags=re.IGNORECASE)
64     clean_text = text[0].strip()
65
66     # Creazione del JSON
67     title_match = re.search(r'<title>(.*?)</title>', result.html, re.
68 IGNORECASE)
69     page_title = title_match.group(1).replace(" - Wikipedia", "") if
70 title_match else "title_not_found"
71
72     return {
73         "url": url,
74         "domain": "en.wikipedia.org",
75         "title": page_title,
76         "html_text": result.html,
77         "parsed_text": clean_text
78     }

```

3.3 Progettazione del Frontend

L'interfaccia utente, esposta sulla porta 8004, è stata progettata per fornire un punto di accesso intuitivo all'ecosistema di valutazione, articolandosi in due moduli principali:

- **Gestione delle Richieste (`frontend.py`):** sviluppato anch'esso in FastAPI, agisce da intermediario sfruttando il client asincrono `httpx` per comunicare con il *backend*. Recupera la lista dei domini, interroga i *gold standard* e monitora in tempo reale lo stato dei container (Backend, MariaDB e Ollama). Il modulo coordina tre flussi principali: l'analisi di URL singoli, un pannello CRUD per la gestione interattiva dei dati di riferimento e una *dashboard* per il monitoraggio delle valutazioni.
- **Motore di Rendering (`templates/` e `static/`):** la presentazione dei dati è affidata al motore Jinja2 per la generazione dinamica dei fogli HTML, stilizzati tramite CSS dedicato. L'interfaccia include una vista comparativa a tre colonne per facilitare il raffronto visivo tra HTML grezzo, *gold standard* e pagina *parsata*, esponendo contestualmente le metriche calcolate.

3.4 Gestione dei Dati e Persistenza

Per garantire la persistenza dei risultati e ottimizzare le risorse di rete, la *pipeline* integra un database relazionale MariaDB. Questa strategia di *caching* locale permette di memorizzare l'HTML delle pagine scansionate, evitando richieste HTTP ridondanti in caso di valutazioni successive sullo stesso URL.

Il database viene inizializzato al primo avvio popolando automaticamente le tabelle di sistema leggendo i file JSON contenuti nella directory `gs_data/`. Lo schema relazionale si articola nelle seguenti tabelle (le chiavi primarie sono evidenziate in grassetto):

- **`web_resources` (`url`, `domain`, `title`, `html_text`, `created_at`):** memorizza l'HTML grezzo prelevato durante la scansione, fungendo da anagrafica centrale.
- **`gold_standard` (`url`, `gold_text`, `created_at`):** archivia il testo pulito di riferimento necessario per il calcolo delle metriche di accuratezza. L'attributo `url` funge da chiave esterna.
- **`parsed_pages` (`url`, `parsed_text`, `created_at`):** conserva il testo elaborato e restituito dal *crawler*.
- **`evaluation_results` (`url`, `token_level_metrics`, `llm_judgments`, `evaluated_at`):** archivia i risultati finali. Al momento dell'inizializzazione i campi assumono valore NULL e vengono popolati dinamicamente. Le metriche insiemistiche vengono salvate in `token_level_metrics`, mentre il voto numerico e il *feedback* testuale dell'intelligenza artificiale risiedono nel campo `llm_judgments`.

Bibliografia

- [1] Patrick Lewis et al. «Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks». In: *Advances in Neural Information Processing Systems*. Vol. 33. 2020, pp. 9459–9474.
- [2] Amazon Web Services (AWS). *Cos'è la Retrieval-Augmented Generation (RAG)?* URL: <https://aws.amazon.com/it/what-is/retrieval-augmented-generation/> (visitato il giorno 31/08/2026).
- [3] Docker Inc. *Docker: Accelerated Container Application Development*. <https://www.docker.com/>. Ultimo accesso: 31 agosto 2026.
- [4] Sebastián Ramírez. *FastAPI*. <https://fastapi.tiangolo.com/>. Ultimo accesso: 1 settembre 2026.
- [5] Pydantic Services Inc. *Pydantic: Data validation using Python type hints*. <https://pydantic.dev/docs/validation/latest/get-started/>. Ultimo accesso: 1 settembre 2026.
- [6] Encode OSS. *HTTPX: A next-generation HTTP client for Python*. <https://www.python-httpx.org/>. Ultimo accesso: 1 settembre 2026.